BATCH 9 · BANKING SECURITY

Large-Number Multiplication in an HSM

23I501-DAA PRESENTATION

.



ABHINANDANA - 24I301
GURUVARSHNAA V L - 24I317
NIVARTHNA S - 24I334
PRAVESHA R - 24I342
SHARMISTHA K - 24I376

1 · Problem modelling

Item	Formal statement
Input	Non-negative integers X, Y of n bits (n = 2048 or 4096), stored as m = n/64 limbs; demo: 5678 × 1234, base 10
Output	P = X·Y, exactly 2n bits (unique correct answer)
Objective	Minimise word operations T(n) of the multiplier, over all correct algorithms
Constraints	512 KB RAM · 3,000 signatures/s · constant-time (no secret-dependent branches or addresses)
Type	Computation (function) problem; choosing the algorithm is an optimisation of cost. Not a decision or enumeration problem.

Schoolbook estimate: n^2 single-digit products. 4096 bits ≈ 1,233 decimal digits → ≈ 1.52 million digit ops; in 64-bit limbs: 64^2 = 4,096 limb multiplies.

2 · Data structure analysis

Structure	Digit access	Add	Split / shift by B^m	Space (4096-bit)
Array of 64-bit limbs (chosen)	O(1)	O(n/64) = 64 steps	O(1) pointer views	512 B
Decimal-digit array	O(1)	O(n/3.3) = 1,233 steps	O(1)	≈ 1.2 KB (1 B/digit)
Linked list of limbs	O(m)	O(m), pointer chasing	O(m) to find middle	≈ 1 KB (+ pointers)

Why limbs: contiguous little-endian array, base 2^64 = one hardware MUL per pair, splitting is free, fixed width gives data-independent addresses (needed for constant time), and it is 16× fewer operations than base 10.

Memoising sub-products? Not inside one multiplication: the three Karatsuba subproblems are always distinct. Yes across signatures: per-key constants (Montgomery R^2, CRT values) and exponent window tables g^i are precomputed once; read them by scanning every entry so the lookup stays constant-time.

3 · Design technique: divide and conquer

Write X = a·B^m + b, Y = c·B^m + d, with B^m = half-size base. Then X·Y = ac·B^2m + (ad + bc)·B^m + bd: four half-size products.

Key identity (Karatsuba): (a + b)(c + d) − ac − bd = ad + bc. So only three half-size products are needed: ac, bd, (a+b)(c+d).

Property that makes it work

Independent subproblems of size n/2 (no shared state, no overlap) and a combine step that costs only Θ(n): additions, subtractions and shifts.

Proof of the gain

T(n) = 3T(n/2) + cn gives Θ(n^1.585); the 4-product version T(n) = 4T(n/2) + cn gives $\Theta(n^2)$. For 64 limbs: 3^6 = 729 base products against 4,096.

$z_2 = a \cdot c$

$z_0 = b \cdot d$

$z_1' = (a+b)(c+d)$

3 independent half-size products, then $z_1 = z_1' - z_2 - z_0$ and shifts: Θ(n) combine

4 · Why the other four techniques fail

Greedy

Counterexample: fix the leading digit from 5 × 1 on 5678 × 1234 and the product starts with 5, but the true answer is 7,006,652. Carries from lower columns change top digits, so a committed choice is never safe. No greedy-choice property.

Backtracking

It searches a candidate space; here the answer is computed, not searched. Even 8-digit candidates for the demo number 10^8, and 2^8192 for a 4096-bit product. Nothing to prune.

Dynamic programming

Needs overlapping subproblems. ac, bd and (a+b)(c+d) have different operands at every level, so a table never hits. Tabulating all digit pairs gives n^2 cells: schoolbook again, no gain.

Branch and bound

Needs an objective to bound. The result is a single exact value, so no partial product is better or worse than another, and any bound would require the answer itself. Bound is absent.

5 · Algorithm (Karatsuba)

```
KARATSUBA(x, y, n)  // state: two n-limb numbers
 if n ≤ T: return SCHOOLBOOK(x, y)  // T ≈ 8–32 limbs
 m ← n/2 ; x = a·B^m + b ; y = c·B^m + d
 z2 ← KARATSUBA(a, c, m)
 z0 ← KARATSUBA(b, d, m)
 z1 ← KARATSUBA(a+b, c+d, m+1) − z2 − z0
 return z2·B^2m + z1·B^m + z0  // shifts and carry-adds
```

Recurrence: T(n) = 3T(n/2) + Θ(n), T(T0) = Θ(1). Base case switches to schoolbook, the practical hybrid. Sub-calls may use fixed (m+1)-limb buffers.



6 · Trace: 5678 × 1234 (a=56, b=78, c=12, d=34)

Call	Sub-products	Combine
z2 = K(56, 12)	K(5,1)=5 · K(6,2)=12 · K(11,3)=33 → mid 33−5−12=16	500 + 160 + 12 = 672
z0 = K(78, 34)	K(7,3)=21 · K(8,4)=32 · K(15,7)=105 → mid 105−21−32=52	2100 + 520 + 32 = 2652
K(56+78, 12+34) = K(134, 46)	K(01,00)=0 · K(34,46)=1564 · K(35,46)=1610 → mid 1610−0−1564=46	0 + 4600 + 1564 = 6164
inner of K(34,46)	K(3,4)=12 · K(4,6)=24 · K(7,10)=70 → mid 34	1200 + 340 + 24 = 1564
inner of K(35,46)	K(3,4)=12 · K(5,6)=30 · K(8,10)=80 → mid 38	1200 + 380 + 30 = 1610
z1 = 6164 − 672 − 2652	2840	
Final	672·10^4 + 2840·10^2 + 2652 = 7,006,652 (matches 5678 × 1234)	

Every node makes exactly 3 sub-calls, as the recurrence predicts.

7 · Complexity and method comparison

Master theorem: a=3, b=2, f(n)=Θ(n); n^(log2 3) ≈ n^1.585 dominates f, so case 1 gives Θ(n^1.585). Recursion tree: level i has 3^i nodes of work c·n·(3/2)^i, a geometric series dominated by the 3^(log2 n) leaves.

Best / worst: identical Θ(n^1.585); the data never changes the work. **Practical:** hybrid with schoolbook base. **Space:** Θ(n) scratch.

Method	Time	Wins when
Schoolbook	$\Theta(n^2)$	below ~16–32 limbs
Karatsuba	Θ(n^1.585)	~32–200 limbs (RSA zone)
Toom-Cook-3	Θ(n^1.465)	~200–1,000 limbs
FFT / Schönhage–Strassen	Θ(n log n log log n)	thousands of limbs

Why libraries switch at thresholds: faster methods hide larger constants (extra additions, evaluation and interpolation, more memory), so a method with better exponent loses on small operands. Switch where the cost curves cross.

8 · Changed requirement: constant-time

Where it leaks

(a+b) and (c+d) can carry into an extra bit, so code branches on carry and trims length. The subtraction z1 − z2 − z0 can branch on sign or borrow. Early exits on zero limbs leak too.

The fix, speed kept

Always use fixed (m+1)-limb buffers. Add the carry bit with masks (0 or all-ones), never an if. Fixed recursion depth and same op sequence. Schoolbook base kernel with no zero-skipping. Cost: a few extra limbs per level; still 3 multiplies.

Stack (4096-bit, 64 limbs, T=8): 3 recursion levels. Scratch per level ≈ 4·(n/2+1) limbs, geometric sum ≈ 4n = 256 limbs = 2 KB, plus frames under 1 KB. Total under 4 KB, far inside 512 KB.

Does the technique hold? Yes. Divide and conquer is unchanged; only the combine step is made data-independent. Throughput (3,000 sig/s) is met by cutting multiply time ≈ 20% at 32–64 limbs (measured, slide 11; 5× at 262,144 bits) and using parallel multiplier hardware.

9 · Proof of correctness

Lemma 1 · schoolbook (loop invariant)

After outer iteration i, r holds x·(y_0 + y_1·B + … + y_i·B^i) and all higher limbs are 0. Each inner step computes a_j·y_i + r + c ≤ $(B-1)^2 + 2(B-1) = B^2-1$, which fits in 2 limbs, so the limb/carry split is exact. At i = n−1, r = x·y.

Theorem · Karatsuba returns x·y

Strong induction on n. Base (n ≤ T): Lemma 1.

Step (n = 2m): the three calls are on size m, so by hypothesis they return exactly z0 = xlo·ylo, z2 = xhi·yhi and sa·sb, where xlo + xhi = sa + ca·B^m and ylo + yhi = sb + cb·B^m (the carry bits are exact).

Expanding: (xlo+xhi)(ylo+yhi) = sa·sb + (ca·sb + cb·sa)B^m + ca·cb·B^2m. The masked adds compute exactly this, since a mask is all-ones iff its carry is 1.

Why nothing overflows or underflows

- The expanded product is < 4B^2m, so 2m+1 limbs hold it.
- Subtracting z0 and z2 leaves xlo·yhi + xhi·ylo ≥ 0, so no subtraction underflows; the value is < 2B^2m, still 2m+1 limbs.
- z0 and z2 sit in disjoint ranges r[0..2m) and r[2m..4m). Adding the middle term at offset m cannot overflow r, because xy < B^4m.

Therefore

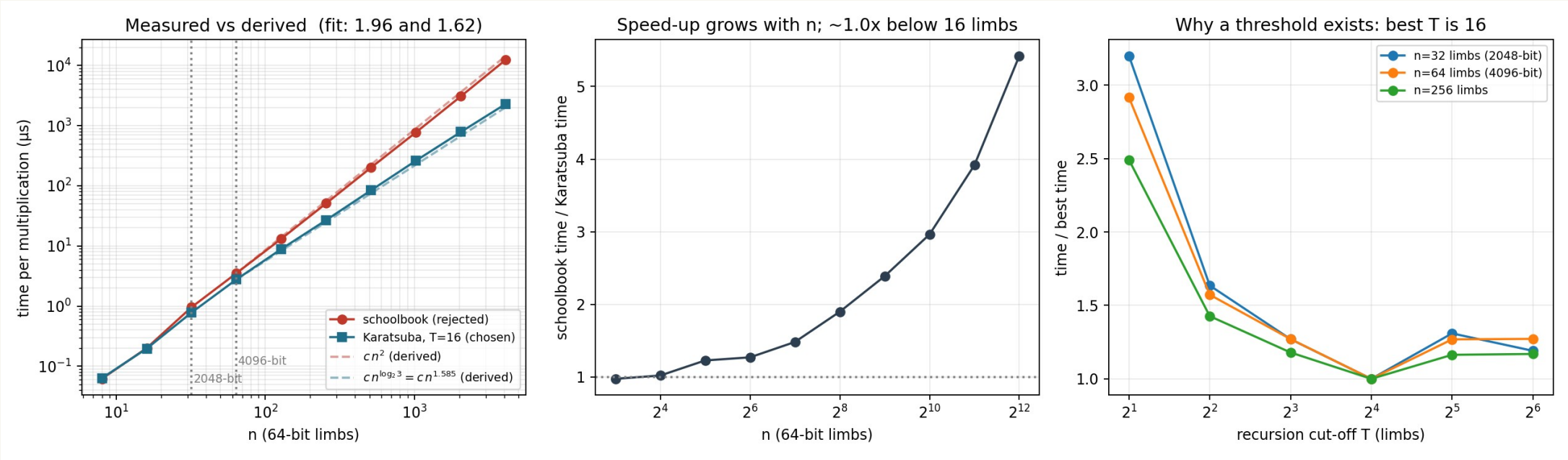
xy = z2B^2m + (xlo·yhi + xhi·ylo)B^m + z0 QED

Termination and pruning

n halves on every call, so recursion ends. No pruning is used: every sub-problem is evaluated, so no optimal solution can be discarded.

Tested: 2,000 cases incl. all-ones and zeros, T = 1…16, 0 mismatches vs schoolbook.

10 · Implementation and empirical analysis



Operand	Limbs	Schoolbook	Karatsuba	Speed-up
2048-bit	32	0.96 µs	0.78 µs	1.23×
4096-bit	64	3.53 µs	2.78 µs	1.27×
16,384-bit	256	51.2 µs	27.0 µs	1.90×
65,536-bit	1,024	777 µs	262 µs	2.96×
262,144-bit	4,096	12.5 ms	2.30 ms	5.42×

Fit vs derivation. Slope on n ≥ 64 limbs: schoolbook 1.965 (derived 2), Karatsuba 1.618 (derived log2 3 = 1.585). The excess over 1.585 is plausibly the Θ(n) additions and memory traffic (not profiled).

Rejected alternative: schoolbook, same file, same inputs. No gain below 16 limbs; the gap widens with n.

Honest limit: 1 shared vCPU, Xeon 2.1 GHz, gcc -O2, best of 7. Ratios are meaningful; absolute times are not an HSM's.

11 · Worst-case input

The input

X = Y = 2^4096 − 1: every limb 0xFF…FF (9999 × 9999 in decimal).

Why it is worst

Every a+b and c+d carries out at every node. Carry census at 64 limbs: 2, 6, 18 carries on levels 0, 1, 2, the maximum possible (random data: 2, 3, 10).

64-digit operands, naive version	Digit products
Best case (1000…0)	729
Random	4,767
All nines (worst)	7,463
Schoolbook, any input	4,096
This code (carry bit kept apart)	**3^k always**

Behaviour. A naive version pads the carried sum to the next size, so sub-problems grow from m to 2m on carry. On all-nines this costs 1.6× the random case, 10× the best case and more than schoolbook, and the run time depends on the data, which is also the timing leak. Trace of 5678 × 1234 done strictly digit by digit: 34 calls and 23 digit products against 16 for schoolbook. This is why slide 7 treats small sums as one machine word.

Fix and effect. Keeping the carry as a separate bit gives exactly 1,728 base-case limb multiplies at 64 limbs (T = 8) on every input. Measured over two runs at 4096 bits: random 2.75–2.79 µs, all-ones 2.67 µs, all-zero 2.51–2.60 µs. Zero is consistently 5–9% faster; no branch in the code explains it, so it needs a dudect-style test before any constant-time claim is signed off.

12 · Real-world reference: OpenSSL and BoringSSL

Aspect	OpenSSL / BoringSSL bn_mul_recursive	This version
Algorithm	Karatsuba, quadratic routine below the cut-over	Karatsuba, schoolbook base case
Cut-over	16 words (BN_MUL_RECURSIVE_SIZE_NORMAL); dedicated 8-word kernel in the OpenSSL tree	16 limbs, tuned by sweep (best at 32, 64, 256 limbs)
Middle term	(a0 − a1)(b1 − b0) + a0·b0 + a1·b1; sign fixed with masks	(a+b)(c+d) − ac − bd, carry bits ca, cb masked in
Operand growth	None: differences never exceed m words	None: carry held as a bit
Sizes	Power-of-two word count n2; separate paths for unequal lengths	Even n only; odd n falls to schoolbook
Scratch	4·n2 words	4n + 64 limbs

The library variant uses the subtractive form, so no operand grows and no carry fix-up is needed, at the price of a sign to handle. Ours keeps the additive form of the lecture identity and pays for it with two masked carry corrections. Both give three half-size multiplications, so the same Θ(n^1.585).